TORÈ — SPECIFICATION PRODUIT ULTRA EXHAUSTIVE

OS numérique des comités, bars et guindaille liégeoise

Document technique complet — prêt développement / sécurité / UX / data

TORÈ — SPECIFICATION PRODUIT (BLOC 1)

Version de travail: 2026-02-14 — Bloc 1/4 (Vision + Architecture + State Machine)

PARTIE 1 — VISION & ARCHITECTURE

1. Vision stratégique

- TORÈ est un OS opérationnel territorial pour la guindaille liégeoise : bars, comités, événements, folklore.

- Objectif produit : devenir **indispensable** (workflow terrain), **non substituable** (dépendances + mémoire), **audit-able** (ledger + logs).

- Postulat terrain : bruit, pénombre, rush, réseau instable, staff fatigué, public alcoolisé → UX “zéro réflexion”.

- Principe : warnings + plafonds + responsabilités explicites plutôt que blocage arbitraire.

- Principe : **un seul point d’entrée** = Scanner QR universel, qui route vers le bon module.

2. Architecture modulaire

- Les entités (Bar, Event, Comité, Groupe) activent/désactivent des modules indépendants.

- Module = UI + API + règles + logs + exports.

- Dépendances explicites : certains modules nécessitent d’autres (ex: Orders nécessite Menu; Bets nécessite Wallet ou Tickets).

- Modes: **USER app** (expérience soirée), **Staff/ORGA app** (opérations terrain), **Portail Web** (ops & admin), **Admin Global** (sécurité/abuse).

3. Modules ON/OFF

- État module: OFF (invisible côté USER), ON (actif), FROZEN (visible mais actions bloquées), READ_ONLY (consultation).

- Toggle gouverné par permissions serveur; toute bascule génère un log signé.

- OFF côté USER = disparition totale (pas de placeholder).

- Règle de priorité : l’**Event State** peut forcer READ_ONLY/FROZEN même si module ON.

4. Ledger append-only

- Toute valeur (tickets, top-up, gifts, refunds, payouts, bets, ardoise) est écrite dans un ledger append-only.

- Une correction n’efface jamais : elle écrit une transaction inverse (compensating entry).

- Chaque entry: id, entity_scope, actor_id, action_type, amount, currency_unit, reason_code, reason_text, refs (order_id/bet_id), timestamp, signature.

- Exigences: idempotence (anti double), ordering stable, exportable CSV/PDF, vérification signature.

5. Scanner QR universel

- Un seul composant scanner (mobile) route vers : join event, menu, order confirm, staff entry, staff bar, bet, hunt, wifi, safe kit.

- QR sensibles signés + expiration + anti-replay + binding context (event_id/bar_id).

- QR non sensible (ex menu public) peut être non signé mais versionné.

- Fallback: saisie code manuel, et écran “QR inconnu” (support + diagnostic).

6. Mode réseau faible

- Caches : menu, infos event, modules actifs, derniers prix, consignes.

- Politique actions :

- Non sensibles → queue locale (ex: log local, check-in boisson) puis sync.

- Sensibles valeur/accès → bloquées si backend down (sauf règles explicites ORGA).

- UI doit exposer: ONLINE / DEGRADED / OFFLINE, avec message actionnable.

7. Architecture multi-tenant (comités isolés)

- Séparation stricte : committee_id est clé de scoping pour données comité (forum, baptême, finances).

- Cross-tenant interdit par défaut; exceptions : Admin Global, statistiques anonymisées, marketplace de bars.

- Contrôle d'accès : ABAC/RBAC hybride (role + scope + module + event_state).

- Rétention et exports RGPD : suppression user, anonymisation logs (selon obligation légale), export complet.

PARTIE 2 — STATE MACHINE GLOBALE

0. Table de priorité des états (global)

| Priorité | État | Scope | Effet UI | Effet API |

|---|---|---|---|---|

| P0 | SECURITY_LOCKDOWN | Global | Tout en lecture seule sauf Admin Global | Refus actions sensibles + exigences raison |

| P1 | BACKEND_DOWN | Global | Bannières offline + routes limitées | Read endpoints cacheables only |

| P2 | EVENT_CLOSED | Event | Event Hub en mode recap | No new orders/bets/entry; exports allowed |

| P3 | EVENT_PAUSE | Event | Actions gelées + message orga | Queue ou refuse selon type |

| P4 | NETWORK_DEGRADED | Device/Event | UI simplifiée + refresh réduit | Backoff + fallback cache |

| P5 | MODULE_FROZEN | Module | CTA désactivés + raison visible | Write endpoints de module refusés |

| P6 | MODULE_ON | Module | Normal | Normal |

1. Event lifecycle

Transitions — Event

| État actuel | Action | État suivant | Autorisé par | Log généré |

|---|---|---|---|---|

| DRAFT | Publier event | SCHEDULED | Owner/Président | event.publish |

| SCHEDULED | Ouvrir (start) | LIVE | Manager/Président | event.open |

| LIVE | Mettre en pause | PAUSE | Manager/Président | event.pause{reason} |

| PAUSE | Reprendre | LIVE | Manager/Président | event.resume |

| LIVE | Basculer MENU_ONLY | MENU_ONLY | Manager | event.menu_only{reason} |

| MENU_ONLY | Revenir LIVE | LIVE | Manager | event.live |

| LIVE | Clôturer event | CLOSED | Owner/Président | event.close{reason} |

| CLOSED | Réouvrir (exception) | LIVE | Owner/Admin Global | event.reopen{reason} |

2. User lifecycle

Transitions — User

État actuel	Action	État suivant	Autorisé par	Log généré
ANON	Créer compte / login	ACTIVE	User	user.activate
ACTIVE	Join event via QR	IN_EVENT	User + Event ON	user.join_event
IN_EVENT	Quitter event	ACTIVE	User	user.leave_event
ACTIVE/IN_EVENT	Demander suppression RGPD	PENDING_DELETE	User	user.delete_request
PENDING_DELETE	Confirmer suppression	DELETED	User	user.delete_confirm
ACTIVE/IN_EVENT	Sanction	SUSPENDED	Admin Global	user.suspend{reason}
SUSPENDED	Lever sanction	ACTIVE	Admin Global	user.unsuspend

3. Baptême lifecycle

Transitions — Baptême

État actuel	Action	État suivant	Autorisé par	Log généré
OFF	Activer module baptême	SETUP	Président/Toge	bapteme.enable
SETUP	Ouvrir inscriptions	REG_OPEN	Encadrant/Toge	bapteme.reg_open
REG_OPEN	Clôturer inscriptions	REG_CLOSED	Encadrant	bapteme.reg_close
REG_CLOSED	Assigner parrains/marraines	ASSIGNED	Encadrant	bapteme.assign
ASSIGNED	Démarrer terrain	FIELD_LIVE	Encadrant	bapteme.field_start
FIELD_LIVE	Déclarer fin	FIELD_CLOSED	Encadrant	bapteme.field_close
FIELD_CLOSED	Archiver année	ARCHIVED	Président	bapteme.archive

4. Order lifecycle

Transitions — Order

État actuel	Action	État suivant	Autorisé par	Log généré
CART	Valider panier	PENDING_QR	User	order.create
PENDING_QR	Afficher QR au bar	AWAITING_SCAN	User	order.present_qr
AWAITING_SCAN	Scan barman	ACCEPTED	Staff Bar	order.accept{station}
ACCEPTED	Marquer en préparation	IN_PROGRESS	Staff Bar	order.in_progress
IN_PROGRESS	Marquer prêt	READY	Staff Bar	order.ready
READY	Remis au client	FULFILLED	Staff Bar	order.fulfilled
AWAITING_SCAN/ACCEPTED	Annuler	CANCELLED	Staff Bar/Manager	order.cancel{reason}
FULFILLED	Refund (exception)	REFUNDED	Manager/Treasurer	order.refund{reason} + ledger.entry

5. Bet lifecycle

Transitions — Bet

État actuel	Action	État suivant	Autorisé par	Log généré
DRAFT	Publier bet	OPEN	Resolver/Manager	bet.publish
OPEN	Placer bet	OPEN	User	bet.place + ledger.entry
OPEN	Fermer marché	LOCKED	Resolver	bet.lock
LOCKED	Résoudre	RESOLVED	Resolver	bet.resolve + ledger.payout
RESOLVED	Ouvrir dispute	DISPUTED	User (window)	bet.dispute_open{reason}
DISPUTED	Décision resolver	RESOLVED_FINAL	Resolver+Manager	bet.dispute_decide{reason}
OPEN/LOCKED	Annuler	CANCELLED	Resolver+Manager	bet.cancel{reason} + ledger.refund

6. Incident lifecycle

Transitions — Incident

État actuel	Action	État suivant	Autorisé par	Log généré
NEW	Créer report	TRIAGE	User/Staff	incident.create
TRIAGE	Assigner owner	ASSIGNED	Manager/Encadrant	incident.assign
ASSIGNED	Marquer en cours	IN_PROGRESS	Assignee	incident.progress
IN_PROGRESS	Résoudre	RESOLVED	Assignee	incident.resolve{reason}
RESOLVED	Exporter dossier	RESOLVED	Manager/Admin Global	incident.export
ANY	Escalader	ESCALATED	Manager	incident.escalate{reason}

7. Battle lifecycle

Transitions — Battle

État actuel	Action	État suivant	Autorisé par	Log généré
CREATED	Ouvrir room	JOIN_OPEN	Manager/Encadrant	battle.open
JOIN_OPEN	Join via QR	JOIN_OPEN	User	battle.join
JOIN_OPEN	Start	LIVE	Arbitre/Resolver	battle.start
LIVE	Submit résultat	PENDING_VALIDATE	Arbitre	battle.result_submit
PENDING_VALIDATE	Valider	CLOSED	Resolver	battle.validate + ranking.update
LIVE/PENDING_VALIDATE	Annuler	CANCELLED	Manager	battle.cancel{reason}

8. Hunt lifecycle

Transitions — Hunt

État actuel	Action	État suivant	Autorisé par	Log généré
DRAFT	Publier hunt	LIVE	Manager/Président	hunt.publish
LIVE	Scan QR clue	LIVE	User	hunt.scan_clue

| LIVE | Claim | CLAIMED | User (first) | hunt.claim + ledger.payout |

| CLAIMED | Dispute | DISPUTED | User | hunt.dispute{reason} |

| DISPUTED | Décider | CLOSED | Resolver | hunt.dispute_decide |

| LIVE | Clôturer | CLOSED | Manager | hunt.close{reason} |

9. Wi-Fi access lifecycle

Transitions — Wi-Fi

| État actuel | Action | État suivant | Autorisé par | Log généré |

|---|---|---|---|---|

| OFF | Activer Wi-Fi module | CONFIG | Owner/Manager | wifi.enable |

| CONFIG | Publier SSID rules | LIVE | Manager | wifi.publish |

| LIVE | User scan QR Wi-Fi | TOKEN_ISSUED | User | wifi.token_issue |

| TOKEN_ISSUED | Connexion réussie | CONNECTED | Captive portal/AP | wifi.connected |

| CONNECTED | Timeout | EXPIRED | System | wifi.expire |

| LIVE | Revoke token | REVOKED | Manager/Admin | wifi.revoke{reason} |

Contrats d’interface (API) — invariants transverses

- Tous les endpoints write requièrent: `X-Idempotency-Key`, `actor_id`, `scope`.
- Tous les logs critiques requièrent: `reason_code` + `reason_text` quand sensible.
- Timestamp serveur authoritative; device time seulement informatif.
- Toute action bloquée doit renvoyer: code erreur, message UX court, et `next_allowed_actions`.

TORÈ — SPECIFICATION PRODUIT (BLOC 2)

Bloc 2/4 — Écrans USER + ORGA EVENT + BAR MODE (objectif: 50+ écrans)

PARTIE 4 — ÉCRANS (Section: USER + ORGA EVENT + BAR MODE)

> Règle: modules OFF invisibles côté USER. Toute action sensible exige confirmation forte + raison obligatoire.

Splash / Boot

1. Nom écran

- Splash / Boot

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Splash / Boot’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App lancée (cold start / warm start)
- Aucun login requis
- Assets locaux disponibles

5. Éléments UI détaillés (liste exhaustive)

- Logo TORÈ centré + glow rubis
- Loader fin jaune (progress/indeterminate) en bas
- Version build + environnement (prod/stage) discret [ASSUMPTION]
- Bouton ‘Diagnostic’ long-press (5s) [ASSUMPTION]
- Écran noir profond + grain léger [ASSUMPTION]

6. Actions possibles (boutons + effets)

- Aucune action (sauf Diagnostic long-press)
- Retry auto si crash init
- Si init > 8s: bouton ‘Relancer’ [ASSUMPTION]

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- app.boot.start
- app.boot.end{duration_ms}
- app.boot.fail{reason}

9. Dépendances backend

- Config fetch (feature flags)

- Auth refresh token (si session)
- Status endpoint (health)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun

Consentement & Charte (join)

1. Nom écran
- Consentement & Charte (join)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Consentement & Charte (join)' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Login / Création profil

1. Nom écran
- Login / Création profil

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Login / Création profil' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Home / Hub

1. Nom écran
- Home / Hub

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l’usage opérationnel de ‘Home / Hub’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

QR Scanner universel

1. Nom écran
- QR Scanner universel

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Scanner un QR et router vers le module exact (menu/join/staff/order/bet/hunt/wifi/safe).

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Caméra plein écran + frame rubis

- Texte guidance minimal (‘Scanne un QR TORÈ’)
- Bottom sheet: lampe, saisie code manuel, aide
- Indicateur anti-replay (icône bouclier) quand QR signé
- Mode faible lumière: auto-torch + toggle manuel
- Zone debug cachée (latence, fps) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- Tap lampe: torch on/off
- Tap ‘Code’: ouvre saisie alphanum
- Tap ‘Aide’: ouvre FAQ scanner
- Scan détecté: haptique + freeze 300ms + route

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}
- qr.scan{qr_type, signed, expires_in_s}
- qr.route{target, success}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- QR verification service (signature/expiry/anti-replay)
- Router service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)
- QR screenshot: si sensible → expiry courte + nonce
- Caméra refusée → fallback code manuel + instructions OS

11. Export lié (si applicable)

- Aucun (par défaut)

QR Résultat — Router

1. Nom écran

- QR Résultat — Router

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘QR Résultat — Router’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Bars — grille cards

1. Nom écran

- Bars — grille cards

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Bars — grille cards’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Bar — détails + events

1. Nom écran

- Bar — détails + events

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Bar — détails + events' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Events (d'un bar)

1. Nom écran
- Events (d'un bar)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Events (d'un bar)' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Join Event

1. Nom écran

- Join Event

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Join Event' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Event Hub

1. Nom écran

- Event Hub

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Event Hub' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Event Annonce / Broadcast

1. Nom écran

- Event Annonce / Broadcast

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Event Annonce / Broadcast’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Menu Event — liste

1. Nom écran

- Menu Event — liste

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Menu Event — liste’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Menu Event — item détail

1. Nom écran

- Menu Event — item détail

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Menu Event — item détail’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Panier

1. Nom écran

- Panier

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Panier’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Confirmation commande

1. Nom écran

- Confirmation commande

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Confirmation commande' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Commande — QR à présenter

1. Nom écran
- Commande — QR à présenter

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Commande — QR à présenter' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Commande — statut live

1. Nom écran

- Commande — statut live

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Commande — statut live' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Wallet — solde

1. Nom écran

- Wallet — solde

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Wallet — solde' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Wallet — historique

1. Nom écran

- Wallet — historique

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Wallet — historique’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Top-up (réception via staff)

1. Nom écran

- Top-up (réception via staff)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Top-up (réception via staff)’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Gift P2P — envoyer

1. Nom écran

- Gift P2P — envoyer

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Gift P2P — envoyer’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Gift P2P — recevoir

1. Nom écran

- Gift P2P — recevoir

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Gift P2P — recevoir’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Ticker Live

1. Nom écran

- Ticker Live

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Ticker Live' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Ticker — filtres

1. Nom écran
- Ticker — filtres

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Ticker — filtres' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Polymarket Global — liste

1. Nom écran

- Polymarket Global — liste

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Polymarket Global — liste' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Polymarket Event — liste

1. Nom écran

- Polymarket Event — liste

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Polymarket Event — liste' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Bet Detail

1. Nom écran

- Bet Detail

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Bet Detail’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Bet Placement

1. Nom écran

- Bet Placement

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Bet Placement’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Dispute Bet

1. Nom écran

- Dispute Bet

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Dispute Bet’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Classement

1. Nom écran

- Classement

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘Classement’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l’écran n’est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Profil

1. Nom écran

- Profil

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Profil' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

Settings — privacy

1. Nom écran
- Settings — privacy

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'Settings — privacy' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- App installée + permissions caméra (scanner) si applicable
- Profil ACTIVE (sauf Splash/Consentement/Login)
- Contexte scope: Global / Bar / Event selon route QR
- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Settings — accessibilité

1. Nom écran

- Settings — accessibilité

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Settings — accessibilité' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

Offline Center

1. Nom écran

- Offline Center

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'Offline Center' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- App installée + permissions caméra (scanner) si applicable

- Profil ACTIVE (sauf Splash/Consentement/Login)

- Contexte scope: Global / Bar / Event selon route QR

- Modules OFF invisibles: l'écran n'est accessible que si module ON ou via route explicite

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Event Control Hub

1. Nom écran

- ORGA — Event Control Hub

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Event Control Hub’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Modules Toggles

1. Nom écran

- ORGA — Modules Toggles

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Modules Toggles’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Urgence / Freeze Center

1. Nom écran

- ORGA — Urgence / Freeze Center

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Urgence / Freeze Center’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)

- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran

- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]

- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Entry Staff (rush scan)

1. Nom écran

- ORGA — Entry Staff (rush scan)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Scanner des entrants en rush avec feedback instantané et compteur géant.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Layout plein écran sans scroll
- Header compact: nom event + badge LIVE + heure + ■■
- Compteur géant ENTRÉES (Space Mono) + delta 10min + cap/%
- Zone scan plein écran + frame
- Barre état: ONLINE/DEGRADED/OFFLINE + queue size
- Log minimal 3 dernières validations (ok/deny)

6. Actions possibles (boutons + effets)

- Scan entrant: flash vert 100–150ms + bip/haptique
- Scan refusé: flash rouge + raison (expiré/déjà entré/banni)
- Tap ■■: ouvre settings (lamp, mode manuel, sons)
- Long-press compteur: bascule ‘mode discret’ (cacher chiffres) [ASSUMPTION]

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}
- entry.scan{ticket_id, outcome, reason_code}
- entry.counter.update{value}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Entry validation service

- Offline queue sync

- Ban list service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

- Double scan même ticket → idempotence sur ticket_id

- Offline: queue locale + conflit lors resync (résolution: first_seen_ts) [ASSUMPTION]

11. Export lié (si applicable)

- Export attendance (CSV/PDF) via ORGA Exports Event

ORGA — Entry Staff (manual override)

1. Nom écran

- ORGA — Entry Staff (manual override)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Entry Staff (manual override)' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)

- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran

- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]

- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Entry Logs

1. Nom écran

- ORGA — Entry Logs

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Entry Logs' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran
- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Rentabilité Live

1. Nom écran

- ORGA — Rentabilité Live

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Rentabilité Live' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran
- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ORGA — Coûts Event (input)

1. Nom écran
- ORGA — Coûts Event (input)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- ORGA

3. Objectif
- Permettre l’usage opérationnel de ‘ORGA — Coûts Event (input)’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Menu Admin (simplifié)

1. Nom écran

- ORGA — Menu Admin (simplifié)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Menu Admin (simplifié)’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Stock & Ruptures (event)

1. Nom écran

- ORGA — Stock & Ruptures (event)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Stock & Ruptures (event)’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)

- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran

- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]

- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ORGA — Broadcast Message

1. Nom écran
- ORGA — Broadcast Message

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- ORGA

3. Objectif
- Permettre l’usage opérationnel de ‘ORGA — Broadcast Message’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès
- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ORGA — Polymarket Admin (liste)

1. Nom écran
- ORGA — Polymarket Admin (liste)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- ORGA

3. Objectif
- Permettre l'usage opérationnel de 'ORGA — Polymarket Admin (liste)' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès
- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran
- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Create Bet

1. Nom écran

- ORGA — Create Bet

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Create Bet' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)

- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran

- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]

- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Resolve Bet

1. Nom écran

- ORGA — Resolve Bet

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Resolve Bet' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran
- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Dispute Queue

1. Nom écran

- ORGA — Dispute Queue

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l'usage opérationnel de 'ORGA — Dispute Queue' dans le scope ORGA avec contraintes terrain.

4. Conditions d'accès

- Rôle staff valide dans l'Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l'écran
- Device mode 'Staff' activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Incident Console (event)

1. Nom écran

- ORGA — Incident Console (event)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Incident Console (event)’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Exports Event

1. Nom écran

- ORGA — Exports Event

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Exports Event’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)
- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran
- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]
- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ORGA — Staff Management (event)

1. Nom écran

- ORGA — Staff Management (event)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ORGA

3. Objectif

- Permettre l’usage opérationnel de ‘ORGA — Staff Management (event)’ dans le scope ORGA avec contraintes terrain.

4. Conditions d’accès

- Rôle staff valide dans l’Event scope (QR rôle ou assignation console)

- Event en SCHEDULED/LIVE/PAUSE/MENU_ONLY selon l’écran

- Device mode ‘Staff’ activé (pin/biométrie) [ASSUMPTION]

- Connexion backend requise sauf écrans explicitement offline queue

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAR — Dashboard

1. Nom écran
- BAR — Dashboard

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif
- Permettre l’usage opérationnel de ‘BAR — Dashboard’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès
- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAR — Orders Live

1. Nom écran
- BAR — Orders Live

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif
- Lister commandes live et permettre traitement rapide (accept/in_progress/ready).

4. Conditions d'accès
- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: Bar name + shift badge + filtre (All/New/Ready)
- KPI strip: commandes en attente, temps moyen, backlog
- Liste dense: item = #order, heure, items, total, status chip, flag identité si opt-in
- Swipe actions (si autorisé): Accept / Ready / Cancel
- Auto-refresh avec backoff (DEGRADED)
- Bouton 'Scan QR' flottant

6. Actions possibles (boutons + effets)
- Tap order: ouvre Order Detail
- Swipe accept: passe à ACCEPTED (confirm si rupture)
- Tap Scan QR: ouvre Scan Order QR
- Tap Cancel: modal raison obligatoire + option refund

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

- order.list.view{filter}

- order.status.change{from,to}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Orders service (list/patch)

- Inventory service

- Ledger (refund)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

- Burst 200 commandes: pagination + virtualized list

- Rupture: conflit stock multi-staff → lock optimiste + retry

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Order Detail

1. Nom écran

- BAR — Order Detail

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l'usage opérationnel de 'BAR — Order Detail' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id

- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]

- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran

- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Scan Order QR

1. Nom écran

- BAR — Scan Order QR

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Scan Order QR’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]

- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Full Menu Admin

1. Nom écran

- BAR — Full Menu Admin

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Full Menu Admin’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Product Editor

1. Nom écran

- BAR — Product Editor

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Product Editor’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Category Manager

1. Nom écran

- BAR — Category Manager

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Category Manager’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Market Settings (bourse)

1. Nom écran

- BAR — Market Settings (bourse)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l'usage opérationnel de 'BAR — Market Settings (bourse)' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Zones & QR Packs

1. Nom écran

- BAR — Zones & QR Packs

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l'usage opérationnel de 'BAR — Zones & QR Packs' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAR — Staff Roles (bar)

1. Nom écran
- BAR — Staff Roles (bar)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif
- Permettre l'usage opérationnel de 'BAR — Staff Roles (bar)' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès
- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAR — Analytics

1. Nom écran
- BAR — Analytics

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif
- Permettre l'usage opérationnel de 'BAR — Analytics' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès
- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Payouts / Cash-out

1. Nom écran

- BAR — Payouts / Cash-out

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Payouts / Cash-out’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]

- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAR — Ardoise (tab) Settings

1. Nom écran
- BAR — Ardoise (tab) Settings

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Ardoise (tab) Settings’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Refunds (exception)

1. Nom écran

- BAR — Refunds (exception)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Permettre l’usage opérationnel de ‘BAR — Refunds (exception)’ dans le scope BAR avec contraintes terrain.

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAR — Wi-Fi Module Setup

1. Nom écran

- BAR — Wi-Fi Module Setup

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- BAR

3. Objectif

- Configurer l’accès Wi■Fi via QR (tokens temporaires, quotas, revoke).

4. Conditions d’accès

- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Form SSID (read-only si existant) + mode (open/captive/token) [ASSUMPTION]
- Durée token (min) + quota device par user
- Bouton ‘Générer QR Wi■Fi’ (pack)
- Liste tokens actifs + revoke
- Stat: connexions, taux échec
- Avertissement légal (pas surveillance, usage limité) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- Save config: confirmation + log
- Generate QR: export PDF pack
- Revoke token: raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}
- wifi.config.update
- wifi.qr.generate
- wifi.token.revoke{reason}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Wi-Fi token service
- QR generator
- Policy/quota service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Export QR Pack Wi■Fi (PDF)

BAR — Safe Kit Setup

1. Nom écran
- BAR — Safe Kit Setup

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- BAR

3. Objectif
- Permettre l'usage opérationnel de 'BAR — Safe Kit Setup' dans le scope BAR avec contraintes terrain.

4. Conditions d'accès
- Compte Bar (Owner/Manager/Staff Bar) + tenant bar_id
- Accès via Portail Web ou App Staff selon écran [ASSUMPTION]
- Modules Bar activés (Menu/Orders/Wi-Fi/Safe Kit…) selon écran
- 2FA obligatoire pour actions financières (payout/refund) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

TORÈ — SPECIFICATION PRODUIT (BLOC 3)

Bloc 3/4 — COMITÉ OS + Baptême + Extensions Liège + Admin Global (écrans)

PARTIE 4 — ÉCRANS (Section: COMITÉ OS, BAPTÊME, EXTENSIONS LIÈGE, ADMIN GLOBAL)

COMITÉ — Landing publique (page comité)

1. Nom écran

- COMITÉ — Landing publique (page comité)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l’usage opérationnel de ‘COMITÉ — Landing publique (page comité)’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Espace privé (home)

1. Nom écran

- COMITÉ — Espace privé (home)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Espace privé (home)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Rôles & permissions (mapping)

1. Nom écran

- COMITÉ — Rôles & permissions (mapping)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Rôles & permissions (mapping)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Staff Directory

1. Nom écran

- COMITÉ — Staff Directory

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Staff Directory' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

COMITÉ — Forum (catégories)

1. Nom écran
- COMITÉ — Forum (catégories)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l'usage opérationnel de 'COMITÉ — Forum (catégories)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

COMITÉ — Thread détail

1. Nom écran
- COMITÉ — Thread détail

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l'usage opérationnel de 'COMITÉ — Thread détail' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Create post

1. Nom écran

- COMITÉ — Create post

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Create post' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Archives annuelles

1. Nom écran

- COMITÉ — Archives annuelles

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Archives annuelles' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Gestion événements (liste)

1. Nom écran

- COMITÉ — Gestion événements (liste)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Gestion événements (liste)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Create event (template 120s)

1. Nom écran

- COMITÉ — Create event (template 120s)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Créer un event en <120s (template) et générer pack QR complet.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Wizard 3 étapes: Infos, Modules, QR/Staff

- Étape 1: nom, lieu, horaires, description courte, mode (ALL-IN vs BAR MODE)

- Étape 2: toggles modules (Menu, Wallet, Orders, Bets, Battles, Hunts, Safe Kit, Wi■Fi)

- Étape 3: assign roles rapides (Entry/Bar/Resolver/Treasurer) + génération QR pack

- Résumé final: preview Event Hub (user) + Control Hub (orga) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- Next/Back avec validation inline

- Create: confirmation forte + log

- Download QR Pack: PDF

- Invite staff: envoi QR rôle (durée) [ASSUMPTION]

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

- committee.event.create

- qr.pack.generate

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Export QR Pack Event (PDF)

COMITÉ — QR Pack Manager

1. Nom écran

- COMITÉ — QR Pack Manager

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — QR Pack Manager' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Gestion modules event

1. Nom écran

- COMITÉ — Gestion modules event

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Gestion modules event' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Broadcast Center

1. Nom écran

- COMITÉ — Broadcast Center

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Broadcast Center' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Finances (budget)

1. Nom écran

- COMITÉ — Finances (budget)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Finances (budget)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

COMITÉ — Ledger comité (viewer)

1. Nom écran

- COMITÉ — Ledger comité (viewer)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'COMITÉ — Ledger comité (viewer)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

COMITÉ — Exports comité

1. Nom écran
- COMITÉ — Exports comité

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l'usage opérationnel de 'COMITÉ — Exports comité' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Setup (périodes + charte)

1. Nom écran

- BAPTÊME — Setup (périodes + charte)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'BAPTÊME — Setup (périodes + charte)' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

- Module Baptême ON pour le comité

- Rôle Encadrant/Toge/Président selon action

- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran

- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAPTÊME — Inscriptions (liste)

1. Nom écran
- BAPTÊME — Inscriptions (liste)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l’usage opérationnel de ‘BAPTÊME — Inscriptions (liste)’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès

- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Invitation / QR inscription

1. Nom écran

- BAPTÊME — Invitation / QR inscription

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l’usage opérationnel de ‘BAPTÊME — Invitation / QR inscription’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès

- Module Baptême ON pour le comité

- Rôle Encadrant/Toge/Président selon action

- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran

- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Baptême service (registrations, assignments, attendance)

- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAPTÊME — Profil Bleu (fiche)

1. Nom écran
- BAPTÊME — Profil Bleu (fiche)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l’usage opérationnel de ‘BAPTÊME — Profil Bleu (fiche)’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès
- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Baptême service (registrations, assignments, attendance)

- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Assignment Parrain/Marraine

1. Nom écran

- BAPTÊME — Assignment Parrain/Marraine

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'BAPTÊME — Assignment Parrain/Marraine' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

- Module Baptême ON pour le comité

- Rôle Encadrant/Toge/Président selon action

- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran

- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BAPTÊME — Planning terrain

1. Nom écran
- BAPTÊME — Planning terrain

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- COMITÉ

3. Objectif
- Permettre l’usage opérationnel de ‘BAPTÊME — Planning terrain’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès
- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Baptême service (registrations, assignments, attendance)

- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Checklists encadrants

1. Nom écran

- BAPTÊME — Checklists encadrants

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'BAPTÊME — Checklists encadrants' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

- Module Baptême ON pour le comité

- Rôle Encadrant/Toge/Président selon action

- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran

- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Présences (scan)

1. Nom écran

- BAPTÊME — Présences (scan)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l’usage opérationnel de ‘BAPTÊME — Présences (scan)’ dans le scope COMITÉ avec contraintes terrain.

4. Conditions d’accès

- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BAPTÊME — Incident discret (baptême)

1. Nom écran

- BAPTÊME — Incident discret (baptême)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre un signalement discret, non public, avec escalade et export.

4. Conditions d’accès

- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Form minimal: type incident (santé/harcèlement/sécurité/logistique), gravité, description
- Toggle anonymat (vis-à-vis encadrants) [ASSUMPTION]
- Bouton ‘Appeler contact urgence’ (si activé) [ASSUMPTION]
- Historique perso des reports (privé)
- Bannière: ‘Stop immédiat si malaise’ (charte)

6. Actions possibles (boutons + effets)

- Submit: confirmation + option ‘Je veux être contacté’
- Escalate: réservé Encadrant/Président
- Export dossier: réservé Président/Admin

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}
- bapteme.incident.create
- incident.escalate

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)

- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Export incidents baptême (PDF/CSV)

BAPTÊME — Clôture & Archive

1. Nom écran

- BAPTÊME — Clôture & Archive

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- COMITÉ

3. Objectif

- Permettre l'usage opérationnel de 'BAPTÊME — Clôture & Archive' dans le scope COMITÉ avec contraintes terrain.

4. Conditions d'accès

- Module Baptême ON pour le comité
- Rôle Encadrant/Toge/Président selon action
- Période REG_OPEN/REG_CLOSED/FIELD_LIVE selon écran
- Charte consentement validée (bleu) avant participation [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Baptême service (registrations, assignments, attendance)
- Incident service (discret)

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

LIÈGE — Calendrier guindailles

1. Nom écran

- LIÈGE — Calendrier guindailles

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'LIÈGE — Calendrier guindailles' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Module Liège ON (global ou comité)
- Contenu public accessible sans join event
- Offline partial cache (carte/agenda)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Content CMS service (glossaire/agenda)
- Map tiles service [ASSUMPTION]

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

LIÈGE — Event détail (calendrier)

1. Nom écran
- LIÈGE — Event détail (calendrier)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'LIÈGE — Event détail (calendrier)' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- Module Liège ON (global ou comité)
- Contenu public accessible sans join event
- Offline partial cache (carte/agenda)

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Content CMS service (glossaire/agenda)
- Map tiles service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

LIÈGE — Glossaire folklore

1. Nom écran

- LIÈGE — Glossaire folklore

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'LIÈGE — Glossaire folklore' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Module Liège ON (global ou comité)
- Contenu public accessible sans join event
- Offline partial cache (carte/agenda)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Content CMS service (glossaire/agenda)
- Map tiles service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

LIÈGE — Carte interactive

1. Nom écran

- LIÈGE — Carte interactive

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘LIÈGE — Carte interactive’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Liège ON (global ou comité)
- Contenu public accessible sans join event
- Offline partial cache (carte/agenda)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Content CMS service (glossaire/agenda)
- Map tiles service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

LIÈGE — Itinéraire safe points

1. Nom écran

- LIÈGE — Itinéraire safe points

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘LIÈGE — Itinéraire safe points’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Liège ON (global ou comité)
- Contenu public accessible sans join event
- Offline partial cache (carte/agenda)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Content CMS service (glossaire/agenda)
- Map tiles service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

SAFE KIT — Buddy setup

1. Nom écran

- SAFE KIT — Buddy setup

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘SAFE KIT — Buddy setup’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Opt-in Safe Kit (user)
- Consentements position/contacts (OS)
- Activation event ou global

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Safe kit service (timers, alerts)
- Notification service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

SAFE KIT — Check-in timer

1. Nom écran
- SAFE KIT — Check-in timer

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'SAFE KIT — Check-in timer' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- Opt-in Safe Kit (user)
- Consentements position/contacts (OS)
- Activation event ou global

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Safe kit service (timers, alerts)

- Notification service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

SAFE KIT — Bouton 'Je veux partir'

1. Nom écran

- SAFE KIT — Bouton 'Je veux partir'

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'SAFE KIT — Bouton 'Je veux partir'' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Opt-in Safe Kit (user)

- Consentements position/contacts (OS)

- Activation event ou global

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Safe kit service (timers, alerts)

- Notification service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

SAFE KIT — Contacts urgence

1. Nom écran

- SAFE KIT — Contacts urgence

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'SAFE KIT — Contacts urgence' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Opt-in Safe Kit (user)

- Consentements position/contacts (OS)

- Activation event ou global

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Safe kit service (timers, alerts)
- Notification service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BATTLES — Hub (event)

1. Nom écran
- BATTLES — Hub (event)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l’usage opérationnel de ‘BATTLES — Hub (event)’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès
- Module Battles ON (event)

- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BATTLES — Create battle

1. Nom écran
- BATTLES — Create battle

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘BATTLES — Create battle’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Battles ON (event)
- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BATTLES — Room join (QR)

1. Nom écran

- BATTLES — Room join (QR)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘BATTLES — Room join (QR)’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Battles ON (event)
- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

BATTLES — Live battle timer

1. Nom écran

- BATTLES — Live battle timer

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'BATTLES — Live battle timer' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Module Battles ON (event)
- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BATTLES — Résultat + validation

1. Nom écran
- BATTLES — Résultat + validation

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'BATTLES — Résultat + validation' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- Module Battles ON (event)
- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

BATTLES — Ranking

1. Nom écran
- BATTLES — Ranking

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l’usage opérationnel de ‘BATTLES — Ranking’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès
- Module Battles ON (event)
- Join event requis
- Rôle Arbitre/Resolver pour validation

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Battles service
- Ranking service

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

CHECK-IN BOISSON — Search

1. Nom écran
- CHECK-IN BOISSON — Search

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- USER

3. Objectif
- Permettre l'usage opérationnel de 'CHECK-IN BOISSON — Search' dans le scope USER avec contraintes terrain.

4. Conditions d'accès
- Module Check-in ON (global)
- Join event optionnel
- Privacy settings appliquées (public/amis/privé)

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Drink catalog service
- Checkin service
- Moderation service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

CHECK-IN BOISSON — Drink detail

1. Nom écran

- CHECK-IN BOISSON — Drink detail

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘CHECK-IN BOISSON — Drink detail’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Check-in ON (global)

- Join event optionnel

- Privacy settings appliquées (public/amis/privé)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Drink catalog service

- Checkin service

- Moderation service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l’état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

CHECK-IN BOISSON — Add check-in

1. Nom écran

- CHECK-IN BOISSON — Add check-in

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l’usage opérationnel de ‘CHECK-IN BOISSON — Add check-in’ dans le scope USER avec contraintes terrain.

4. Conditions d’accès

- Module Check-in ON (global)

- Join event optionnel

- Privacy settings appliquées (public/amis/privé)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d’events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Drink catalog service

- Checkin service

- Moderation service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

CHECK-IN BOISSON — Badges

1. Nom écran

- CHECK-IN BOISSON — Badges

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'CHECK-IN BOISSON — Badges' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Module Check-in ON (global)

- Join event optionnel

- Privacy settings appliquées (public/amis/privé)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Drink catalog service

- Checkin service

- Moderation service [ASSUMPTION]

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

CHECK-IN BOISSON — Stats perso

1. Nom écran

- CHECK-IN BOISSON — Stats perso

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- USER

3. Objectif

- Permettre l'usage opérationnel de 'CHECK-IN BOISSON — Stats perso' dans le scope USER avec contraintes terrain.

4. Conditions d'accès

- Module Check-in ON (global)

- Join event optionnel

- Privacy settings appliquées (public/amis/privé)

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Drink catalog service
- Checkin service
- Moderation service [ASSUMPTION]

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ADMIN — Global Dashboard

1. Nom écran
- ADMIN — Global Dashboard

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- ADMIN

3. Objectif
- Permettre l’usage opérationnel de ‘ADMIN — Global Dashboard’ dans le scope ADMIN avec contraintes terrain.

4. Conditions d’accès
- Compte Admin Global
- 2FA obligatoire
- Justification obligatoire (raison) pour toute action
- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Admin policy engine
- KMS/signature verification
- Rate limiter + abuse detection

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ADMIN — Audit Logs Search

1. Nom écran

- ADMIN — Audit Logs Search

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ADMIN

3. Objectif

- Permettre l’usage opérationnel de ‘ADMIN — Audit Logs Search’ dans le scope ADMIN avec contraintes terrain.

4. Conditions d’accès

- Compte Admin Global

- 2FA obligatoire
- Justification obligatoire (raison) pour toute action
- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)
- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)
- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Admin policy engine
- KMS/signature verification
- Rate limiter + abuse detection

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ADMIN — Security Lockdown Control

1. Nom écran

- ADMIN — Security Lockdown Control

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ADMIN

3. Objectif

- Activer un verrouillage sécurité global ou par tenant/module.

4. Conditions d’accès

- Compte Admin Global
- 2FA obligatoire
- Justification obligatoire (raison) pour toute action
- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Toggles: Global lockdown / par comité / par bar / par event
- Actions: set READ_ONLY, freeze module, revoke QR keys
- Champ raison obligatoire + durée (timeboxed)
- Preview impact: endpoints bloqués + écrans affectés
- 4-eyes confirmation (second admin) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- Proposer lockdown: crée demande
- Valider lockdown: second admin confirme
- Rollback: nécessite raison + post-mortem link [ASSUMPTION]

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}
- admin.lockdown.propose
- admin.lockdown.confirm
- admin.lockdown.rollback

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Admin policy engine
- KMS/signature verification
- Rate limiter + abuse detection

- Feature flag service (kill switch)

- Key rotation service

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ADMIN — Sanctions (users/bars/committees)

1. Nom écran

- ADMIN — Sanctions (users/bars/committees)

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ADMIN

3. Objectif

- Permettre l'usage opérationnel de 'ADMIN — Sanctions (users/bars/committees)' dans le scope ADMIN avec contraintes terrain.

4. Conditions d'accès

- Compte Admin Global

- 2FA obligatoire

- Justification obligatoire (raison) pour toute action

- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions

- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions

- error: message court + code + retry + fallback offline si possible

- empty: état vide contextualisé (pas de données / module OFF / pas d'events)

- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}

- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)

- Permissions service (RBAC/ABAC) + module registry

- Event/Bar/Committee service selon contexte

- Logging + Ledger (si valeur/état)

- Admin policy engine

- KMS/signature verification

- Rate limiter + abuse detection

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire

- Désynchronisation state machine: UI doit afficher l'état serveur authoritative

- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes

- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

ADMIN — Data retention & RGPD queue

1. Nom écran

- ADMIN — Data retention & RGPD queue

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)

- ADMIN

3. Objectif

- Permettre l'usage opérationnel de 'ADMIN — Data retention & RGPD queue' dans le scope ADMIN avec contraintes terrain.

4. Conditions d'accès

- Compte Admin Global

- 2FA obligatoire

- Justification obligatoire (raison) pour toute action

- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)

- Zone principale: contenu spécifique écran (list/detail/form)

- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]

- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable

- Accès support: lien 'Aide' + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)

- Secondary CTA (outline rubis) pour annuler/retour

- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)
- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d'events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)
- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend
- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Admin policy engine
- KMS/signature verification
- Rate limiter + abuse detection

10. Risques & edge cases
- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l'état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)
- Aucun (par défaut)

ADMIN — Incident escalations

1. Nom écran
- ADMIN — Incident escalations

2. Contexte (USER / ORGA / BAR / COMITÉ / ADMIN)
- ADMIN

3. Objectif
- Permettre l'usage opérationnel de 'ADMIN — Incident escalations' dans le scope ADMIN avec contraintes terrain.

4. Conditions d'accès
- Compte Admin Global
- 2FA obligatoire
- Justification obligatoire (raison) pour toute action
- Accès journalisé + revu (4-eyes) [ASSUMPTION]

5. Éléments UI détaillés (liste exhaustive)

- Header: titre + scope badge (Global/Bar/Event) + état réseau (ONLINE/DEGRADED/OFFLINE)
- Zone principale: contenu spécifique écran (list/detail/form)
- Feedback immédiat: toast court + haptique (mobile) [ASSUMPTION]
- Bannière état Event (LIVE/PAUSE/MENU_ONLY/CLOSED) quand applicable
- Accès support: lien ‘Aide’ + diagnostic (QR id, version, latence) [ASSUMPTION]

6. Actions possibles (boutons + effets)

- CTA principal (jaune) quand action confirmante (JOIN / CONFIRMER / VALIDER)
- Secondary CTA (outline rubis) pour annuler/retour
- Gestes: pull-to-refresh (si scroll) + swipe (si liste) selon permissions
- Confirmations fortes pour actions sensibles (freeze, refund, resolve bet) avec raison obligatoire

7. États possibles (loading / error / empty / success)

- loading: skeleton + disable actions
- error: message court + code + retry + fallback offline si possible
- empty: état vide contextualisé (pas de données / module OFF / pas d’events)
- success: animation courte + retour état stable + log visible (orga/bar)

8. Logs générés (si applicable)

- ui.view{screen_name, scope, device_state}
- ui.action{action, outcome, latency_ms}

9. Dépendances backend

- Auth service (token + roles + scopes)
- Permissions service (RBAC/ABAC) + module registry
- Event/Bar/Committee service selon contexte
- Logging + Ledger (si valeur/état)
- Admin policy engine
- KMS/signature verification
- Rate limiter + abuse detection

10. Risques & edge cases

- Réseau instable: double tap / double submit → idempotency key obligatoire
- Désynchronisation state machine: UI doit afficher l’état serveur authoritative
- Confusion scope (bar vs event): badge scope obligatoire + couleurs distinctes
- Accessibilité: contraste + tailles texte (mode high contrast)

11. Export lié (si applicable)

- Aucun (par défaut)

TORÈ — SPECIFICATION PRODUIT (BLOC 4)

Bloc 4/4 — Permissions + Exports + KPI/SLO + Roadmap technique

PARTIE 3 — MATRICE PERMISSIONS COMPLÈTE

| Action | Contexte | Description | Owner | Manager | Staff Entry | Staff Bar | Treasurer | Resolver | Music Admin | Président | Toge | Encadrant | Bleu | Ancien | Admin Global |

|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| event.publish | Event | Publier un event | ■ | — | — | — | — | — | ■ | — | — | — | — | ■ |

| event.open_close | Event | Ouvrir/Clôturer event | ■ | ■ | — | — | — | — | ■ | — | — | — | — | ■ |

| event.pause_resume | Event | Pause/Resume/MENU_ONLY | ■ | ■ | — | — | — | — | ■ | — | — | — | — | ■ |

| module.toggle | Event/Bar/Comité | Activer/Freeze un module | ■ | ■ | — | — | — | — | ■ | ■ | — | — | — | ■ |

| entry.validate | Event | Valider entrée (scan) | ■ | ■ | ■ | — | — | — | — | — | — | — | — | — |

| menu.edit_simple | Event | Édit prix/stock (simplifié) | ■ | ■ | — | ■ | — | — | — | — | — | — | — | — |

| menu.edit_full | Bar | Édit menu complet | ■ | ■ | — | ■ | — | — | — | — | — | — | — | — |

| order.accept_fulfill | Bar/Event | Accepter/Servir commande | ■ | ■ | — | ■ | — | — | — | — | — | — | — | — |

| order.cancel_refund | Bar/Event | Annuler/Refund | ■ | ■ | — | — | ■ | — | — | — | — | — | — | — |

| wallet.topup | Event/Bar | Top-up staff | ■ | ■ | — | ■ | ■ | — | — | — | — | — | — | — |

| gift.p2p | Global/Event | Envoyer gift tickets | ■ | ■ | — | — | — | — | — | — | — | — | ■ | ■ | — |

| bet.create_resolve | Event | Créer/Résoudre bet | ■ | ■ | — | — | — | ■ | — | — | — | — | — | — | — |

| bet.dispute_decide | Event | Décider dispute | ■ | ■ | — | — | — | ■ | — | — | — | — | — | — | ■ |

| incident.manage | Event/Comité | Assigner/Résoudre incident | ■ | ■ | — | — | — | — | — | ■ | — | ■ | — | — | ■ |

| exports.generate | All | Générer exports | ■ | ■ | — | — | ■ | — | — | ■ | — | — | — | — | ■ |

| bapteme.manage | Comité | Gérer baptême (setup/assign/presences) | — | — | — | — | — | — | — | — | ■ | ■ | ■ | — | — | ■ |

| wifi.configure | Bar/Event | Configurer Wi■Fi tokens | ■ | ■ | — | — | — | — | — | — | — | — | — | — | — | ■ |

| safe.configure | Bar/Event | Configurer Safe Kit | ■ | ■ | — | — | — | — | — | — | — | — | — | — | — | ■ |

| admin.lockdown | Global | Lockdown sécurité | — | — | — | — | — | — | — | — | — | — | — | — | — | ■ |

PARTIE 6 — EXPORTS EXHAUSTIFS

Rapport fin de soirée (Event)

- Format: PDF
- Nom fichier: `rapport_fin_de_soir_e_event` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Owner/Manager/Treasurer/Président
- Log associé: `export.event.report`

Champs exacts

| Champ | Type | Notes |

|---|---|---|

| event_id | UUID | — |

| event_name | string | — |

| bar_id | UUID | — |

| start_ts | datetime | — |

| end_ts | datetime | — |

| event_state_timeline | json | Transitions horodatées |

| entries_total | int | Compteur final |

| entries_by_interval_10m | json | Historique densité |

| revenue_tickets | decimal | Somme ledger + tickets |

| revenue_orders | decimal | Somme orders payés |

| costs_input | json | Fûts/bouteilles/logistique |

| profit_estimate | decimal | Recettes - coûts |

| top_products | json | Top ventes |

| incidents_summary | json | Liste + gravité |

| signatures | json | hash + sign |

Export finances comité

- Format: CSV+PDF

- Nom fichier: `export_finances_comit` + timestamp

- Horodatage: UTC + offset local affiché

- Signature: hash contenu + signature serveur

- Qui peut générer: Président/Treasurer/Owner

- Log associé: `export.committee.finances`

Champs exacts

| Champ | Type | Notes |

|---|---|---|

| committee_id | UUID | — |

| period_start | date | — |

| period_end | date | — |

| ledger_entries[] | rows | append-only |

| budget_lines[] | rows | catégorie/montant |

| attachments[] | rows | factures [ASSUMPTION] |

| signature | string | hash/sign |

Export baptême

- Format: PDF+CSV

- Nom fichier: `export_bapt_me` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Président/Toge/Encadrant
- Log associé: `export.bapteme`

Champs exacts

Champ	Type	Notes
year	string	2025-2026
registrations[]	rows	bleu_id, consent_ts, status
assignments[]	rows	bleu_id, parrain_id, marraine_id
attendance[]	rows	session_id, bleu_id, presence, ts
incidents[]	rows	id, type, severity, resolution
charter_version	string	—
signature	string	—

Export incidents

- Format: CSV
- Nom fichier: `export_incidents` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Manager/Président/Admin Global
- Log associé: `export.incidents`

Champs exacts

Champ	Type	Notes
incident_id	UUID	—
scope	enum	event/committee/bar
created_ts	datetime	—
reporter_id	UUID/anon	selon anonymat
type	enum	santé/sécurité/harcèlement/logistique
severity	enum	low/med/high/critical
status	enum	triage/assigned/in_progress/resolved
timeline[]	rows	actions+actors+reasons
resolution	text	—

Export ledger

- Format: CSV
- Nom fichier: `export_ledger` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Owner/Treasurer/Admin Global
- Log associé: `export.ledger`

Champs exacts

Champ	Type	Notes
entry_id	UUID	—
scope	string	bar/event/committee
actor_id	UUID	—
action_type	string	—
amount	decimal	—
unit	string	tickets/€
reason_code	string	—
reason_text	string	—
refs	json	order_id/bet_id
ts	datetime	—
signature	string	—

Export bets

- Format: CSV
- Nom fichier: `export_bets` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Resolver/Manager/Admin Global
- Log associé: `export.bets`

Champs exacts

Champ	Type	Notes
bet_id	UUID	—
scope_event_id	UUID	—
type	enum	objective/subjective
question	string	—
state	enum	open/locked/resolved/disputed
pool_total	decimal	—
participants	int	—

| outcome | enum | YES/NO/CANCEL |

| placements[] | rows | user_id, side, amount, ts |

| payouts[] | rows | user_id, amount, ts |

| disputes[] | rows | reason, decision, ts |

Export hunts

- Format: CSV
- Nom fichier: `export_hunts` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Manager/Resolver
- Log associé: `export.hunts`

Champs exacts

| Champ | Type | Notes |

|---|---|---|

| hunt_id | UUID | — |

| event_id | UUID | — |

| clues[] | rows | qr_id, location_hint |

| scans[] | rows | user_id, qr_id, ts |

| winner_id | UUID | — |

| payout | decimal | — |

| disputes | json | — |

Export battles

- Format: CSV
- Nom fichier: `export_battles` + timestamp
- Horodatage: UTC + offset local affiché
- Signature: hash contenu + signature serveur
- Qui peut générer: Resolver/Manager
- Log associé: `export.battles`

Champs exacts

| Champ | Type | Notes |

|---|---|---|

| battle_id | UUID | — |

| event_id | UUID | — |

| format | enum | 1v1/ffa/team |

| participants[] | rows | user_id, team |

| start_ts | datetime | — |

| end_ts | datetime | — |

| result | json | scores/winner |

| validated_by | UUID | — |

Export attendance

- Format: CSV

- Nom fichier: `export_attendance` + timestamp

- Horodatage: UTC + offset local affiché

- Signature: hash contenu + signature serveur

- Qui peut générer: Manager/Owner

- Log associé: `export.attendance`

Champs exacts

| Champ | Type | Notes |

|---|---|---|

| event_id | UUID | — |

| entry_scans[] | rows | ticket_id/user_id?, ts, outcome, reason |

| entries_total | int | — |

| offline_queue_reconciliations | json | — |

PARTIE 7 — KPI & SLO

| Catégorie | Métrique | Définition | Cible | Mesure |

|---|---|---|---|---|

| Activation comité | Time-to-first-event | Temps création comité → event publié | < 10 min | logs committee.create + event.publish |

| Adoption forum | DAU forum | Utilisateurs actifs forum / jour | [ASSUMPTION] 30% des membres | ui.view forum + post |

| Rentabilité live | Erreur estimation profit | \|profit_estimate - real\| / real | < 10% | comparaison post-event (inputs) |

| Fiabilité | Crash-free % | Sessions sans crash | ≥ 99% | crash reporting |

| Scanner | p95 scan-to-route | Temps scan → écran destination | < 1s | qr.scan + ui.view destination |

| Service bar | p95 order accept | Création order → ACCEPTED | < 20s | order.create + order.accept |

| Bets | Temps résolution bet | LOCKED → RESOLVED_FINAL | < 5 min | bet.lock + bet.resolve_final |

| Réseau faible | Taux succès offline queue | Actions non sensibles sync sans conflit | ≥ 98% | sync logs |

PARTIE 8 — ROADMAP PRIORISÉE TECHNIQUE (T1–T6)

T1 — Core platform

- QR universel + Router + verification signatures

- RBAC/ABAC + modules ON/OFF + logs

- Cache réseau faible (menu + event)
- Event templates + QR pack export

Dépendances: Auth, Feature flags, QR service, Logging

T2 — Comités console + entrée

- Entry staff rush mode + offline queue
- Rentabilité live + coûts simples
- Broadcast messages

Dépendances: Event state machine, Attendance service, Export v1

T3 — Bar kit

- Menu QR complet + portail web minimal
- Orders mode C (scan barman) + statuts
- Staff roles bar + analytics v1

Dépendances: Inventory, Orders, Device auth staff

T4 — Tickets & valeur

- Wallet tickets unique + top-up staff + gifts
- Ledger append-only complet + refunds
- Ardoise optionnelle + plafonds soft

Dépendances: Ledger, Anti-fraud, RGPD exports

T5 — Bets/Battles/Hunts

- Polymarket (objective/subjective) + dispute escalation
- Battles/affonds + ranking
- Hunts QR + payouts

Dépendances: Resolver roles, Ledger payouts, Notification

T6 — Comité OS + extensions Liège + Admin global

- Forum, archives, rôles custom
- Module baptême + incident discret
- Calendrier Liège + glossaire + carte + safe kit + wifi
- Admin global: sanctions, lockdown, audits

Dépendances: Multi-tenant isolation, Moderation, KMS, Compliance